

Original Research Article

Adaptive Personalized Federated Learning for Cross-Domain Next-Word Prediction with Dynamic LoRA Rank Masking and Cross-Attention Gated Expert Routing

Kanishkaa Boopathi Raaj, Karthik Pranav Sivaraj, Gokul C*

Department of Computer Science and Engineering, Kongu Engineering College, Perundurai, Erode, Tamil Nadu, India

*Corresponding author. E-mail: [corresponding author e-mail] ORCID: [ORCID iD]

Received [date]; revised [date]; accepted [date]

ABSTRACT

Mobile keyboards need next-word prediction that adapts to how an individual writes, yet the text that enables such adaptation is among the most sensitive data a person produces. Federated learning addresses data transport but not heterogeneity: real clients differ by orders of magnitude in data volume and in language, script and register. We present an adaptive personalized federated learning framework in which each client trains a LoRA adapter whose active rank is derived from a data-volume personalization profile. Local updates are gradient-clipped, perturbed with Gaussian noise and transmitted under a zero-sum pairwise secure-aggregation mask, then combined by a dimension-aware federated averaging rule that tolerates heterogeneous ranks; a Rényi differential privacy accountant reports the budget consumed. On the resulting bank of frozen client experts we train CAGER, a cross-attention gated expert router conditioned on client identity, active rank and personalization score, and we study three variants of increasing client-awareness. The framework is evaluated on a six-corpus multilingual collection (Shakespearean English, Tamil-script news and film reviews, romanized Tamil (Tanglish) and Tamil–English code-mixed text) partitioned across eleven non-IID clients, and is exercised through a keyboard-style three-candidate prediction engine with on-device prefix completion. In a CPU-only feasibility pilot the pipeline completes all stages, including three federated rounds with unified aggregation, at a confirmed $\epsilon = 15.25$ ($\delta = 10^{-5}$). The full system (CAGER v3 + FedProx + LDP) reaches a perplexity of ≈ 11.2 at round 100, against 17.5 for FedProx + LoRA and 23.8 for standard FedAvg, while cumulative network traffic falls from $\sim 50,000$ MB to $\sim 1,050$ MB over 100 rounds, a 97.9% reduction.

Keywords: federated learning; low-rank adaptation; differential privacy; mixture of experts; cross-attention routing; multilingual NLP; next-word prediction; non-IID learning

1. INTRODUCTION

Mobile keyboards collect some of the most revealing text a person produces: passwords typed by mistake, medical searches, private messages, and the names of people and places. The regulatory and ethical case for keeping that text on the device is straightforward. The engineering case for nevertheless training a useful language model on it is harder. Federated learning, introduced in its modern form by McMahan et al. [1] and applied to keyboard prediction by Hard et al. [2], offers a workable compromise: clients train locally and upload parameter updates rather than raw data, so the server never holds the sequences directly.

The compromise is easier to state than to implement. When the backbone is a transformer-based causal language model, full gradient updates can exceed 300 MB per client per round, which is an unrealistic demand on a mobile connection. When eleven clients are drawn from six corpora spanning three scripts and several registers, a simple sample-

weighted average will mix updates that pull the model in directions unrelated to one another, producing a global model that is mediocre at everything [3, 4]. Even stripped-down gradient vectors can be inverted to recover training text, particularly on small local datasets. And a single fixed adapter size treats a client with eighty examples identically to one with twenty-six.

This paper describes a system that addresses all four problems in a unified pipeline. The novelty lies not in any single component but in how the components interact: rank masking preserves aggregation compatibility while privacy noise is stabilized by the proximal anchor, and the expert router exploits the frozen diversity of the client adapters rather than trying to merge it away.

1.1. Contributions

The paper makes five contributions:

- **Rank-adaptive personalized PEFT.** A profiling rule maps client data volume to an active LoRA rank $r_k \in \{4, 8, 12, 16\}$, implemented by

masking a fixed rank-16 adapter so that clients of heterogeneous capacity remain aggregation-compatible.

- **Dimension-aware federated averaging with a full privacy stack.** Sample-weighted aggregation across mismatched active ranks is combined with FedProx anchoring, DP-SGD-style clipped and perturbed local updates, zero-sum pairwise secure aggregation, and a Rényi accountant.
- **CAGER: cross-attention gated expert routing.** A lightweight router attends from the backbone’s hidden states to a context of client identity, active rank and personalization score, and mixes the top-K frozen experts.
- **A multilingual non-IID keyboard benchmark.** Eleven clients drawn from six corpora spanning Tamil script, romanized Tamil, code-mixed Tamil–English and literary English, with a keyboard-style three-candidate inference engine.
- **A negative and diagnostic result.** At a small training budget, routing gains vanish, the router collapses onto its own expert, and GPT-2 byte-level tokenization makes perplexity on Tamil script incomparable to perplexity on Latin script.

The remainder of the paper is organized as follows. Section 2 presents a literature survey; Section 3 gives the system architecture; Section 4 details the method; Sections 5 and 6 describe the experimental setup and results; Section 7 analyzes privacy; Section 8 discusses limitations; and Section 9 concludes. Table 1 summarizes the notation.

TABLE 1. Summary of notation. The symbol α is overloaded: LoRA scale (Section 4.3), Rényi order (Section 4.8) and Dirichlet concentration (Sections 5.4 and 6.5).

Symbol	Meaning
$K; n_k$	Number of clients (11); local training size of client k
D_k, P_k	Private corpus and data distribution of client k
$\theta; \Delta_k$	Frozen backbone weights; client adapter
$s_k; \Gamma_k; \Gamma_{\max}$	Personalization score; active rank; physical rank (16)
$m_k; A, B$	Binary rank mask; LoRA down/up matrices
α	LoRA scaling factor (32)
μ	FedProx coefficient (0.01)
$C; \sigma; B$	Clip norm; noise multiplier; batch size
M_{ij}	Pairwise secure-aggregation mask
$\epsilon; \delta; T; \rho(\alpha)$	Privacy budget; number of steps; per-step RDP cost
$c; e_{id}, e_{rank}, e_{score}$	Router context; identity, rank and score embeddings
$g; p^*; \text{Comp}$	Gate distribution; target distribution; compatibility scorer

2. LITERATURE SURVEY

This section surveys the literature along six themes that correspond to the components of the proposed system: (i) federated learning under statistical heterogeneity, (ii) personalization, (iii) parameter-efficient fine-tuning in federated settings, (iv) privacy mechanisms, (v) mixture-of-experts routing, and (vi) code-mixed and low-resource NLP. Section 2.7 summarizes the surveyed work and Section 2.8 states the research gaps that this paper addresses.

The literature search covered research on federated learning, personalized federated learning, parameter-efficient fine-tuning, differential privacy, secure aggregation, mixture-of-experts routing, and multilingual next-word prediction. Relevant studies were identified using academic search engines and scholarly databases, including Google Scholar, IEEE Xplore, and the ACM Digital Library. Search terms included “federated learning under non-IID data,” “personalized federated learning,” “LoRA for federated learning,” “differentially private language models,” “mixture-of-experts routing,” and “Tamil–English code-mixed language modelling.”

2.1. Federated learning under heterogeneity

Federated averaging [1] established the basic protocol: clients perform local epochs, the server takes a sample-weighted mean of the resulting parameters, and the cycle repeats (see [5] for a survey). Its weakness under non-IID partitions is well documented [3, 4], and the responses fall into several families. Regularization and correction methods such as FedProx [6], SCAFFOLD [7] and FedDyn [8] modify the local objective or update rule to limit client drift; FedNova [9] addresses the objective inconsistency caused by unequal local work; and adaptive server optimizers [10] change how the aggregate is applied. Keyboard prediction has been a flagship application: Hard et al. [2] trained a recurrent language model with federated averaging for on-device next-word prediction. Such systems assume a single shared architecture for all clients, and each client receives the same modelling capacity regardless of how much data it holds.

2.2. Personalization

Personalization approaches [11] accept that a single global model is the wrong goal. Per-FedAvg [12] and pFedMe [13] optimize for fast local adaptation, FedMD [14] and ensemble distillation [15] transfer knowledge between heterogeneous models, and attention-based approaches [16] adapt a global model

to each client. Our framework belongs to the personalization family but retains a shared aggregation path through a FedProx anchor.

2.3. Parameter-efficient fine-tuning in federated settings

Low-rank adaptation (LoRA) [17] learns a rank- r update, which reduces both the trainable parameters and the payload a client must upload. Bottleneck adapters [18], prompt tuning [19], bias-only tuning [20] and few-shot recipes [21] share the goal of freezing the backbone, and low-rank structure has a long history in learning theory [22]. AdaLoRA [23] allocates rank by importance during training. In federated settings, parameter-efficient fine-tuning has been used to cut communication for language models [24], and heterogeneous-rank schemes let clients with different compute budgets train different ranks [25, 26, 27].

In most of these formulations the adapter rank or size is a global hyperparameter fixed before training; the profiling rule of Section 4.2 instead lets it vary per client. We sidestep the shape problem by fixing the physical rank and varying only which columns are active, so every client uploads the same tensor layout and the aggregation rule sees a consistent dimension while the effective capacities still differ.

2.4. Privacy mechanisms

Differential privacy [28] bounds what any output reveals about a single training example. Differentially private stochastic gradient descent [29] clips per-example gradients and adds calibrated Gaussian noise; it has been applied to recurrent language models in the federated setting [30] and to fine-tuning of large language models [31, 32]. The privacy loss over many steps is tracked by a moments or Rényi accountant [33], with the Gaussian mechanism calibrated as in [34]; alternative accounting uses Gaussian differential privacy [35] and the discrete Gaussian [36]. Non-IID data interacts with these guarantees [37].

Two observations motivate the privacy design here. Gradient inversion is easier on small local datasets, which are exactly the clients assigned the lowest ranks in our framework; and secure aggregation protects individual updates from the server but not against inference from the released model. Secure aggregation [38] is complementary rather than redundant: differential privacy bounds what can be inferred from

the aggregate, while secure aggregation prevents the server from seeing any individual update.

2.5. Mixture-of-experts routing

Sparsely gated mixtures [39] route each input to a small subset of experts and have been scaled to very large models [40]. The idea transfers to federated learning if each client’s adapter is treated as an expert, and federated mixtures [41], gated parameter routing [42], dynamic routing adapters [43], and cross-attention gating for domain shift [44, 45] have been proposed for language tasks. The federated version has a property the centralized version does not: the router knows who is asking. CAGER exploits this by making client identity, capacity and profile the key-value context of a cross-attention operation [46].

2.6. Code-mixed and low-resource Indic NLP

Tamil–English code-mixing and romanized Tamil are common in South Indian digital communication and are poorly served by models tokenized for English. Multilingual encoders [47, 48] exist, but generative backbones such as GPT-2 [49] use English-centric byte-level BPE, and federated work on keyboard vocabularies [50] has focused on out-of-vocabulary words rather than script. As discussed in Section 8, byte-level BPE fragmentation does not merely hurt accuracy: it distorts perplexity in a way that can be mistaken for good performance.

2.7. Summary of the surveyed literature

Table 2 condenses the six themes. Each row lists representative works, the idea they establish, and the gap that remains relative to the present study. Table 3 then positions the proposed framework against representative methods; its entries reflect the primary contribution of each cited method, not every extension.

Chronologically, the surveyed work shows a clear progression. Federated averaging [1] (2017) and its first keyboard deployment [2] were followed in 2020–2021 by drift-control, normalization and adaptive-optimizer methods [6, 7, 8, 9, 10] and by personalized formulations [12, 13]. From 2021 onward, parameter-efficient tuning [17, 19, 20] entered the federated setting [24], and 2023–2024 brought heterogeneous-rank and routing-based proposals [25, 26, 41, 42, 43, 44, 45]. Privacy machinery [29, 33, 38] predates most of these developments and, in the surveyed work, is usually treated separately from the architectural contribution.

TABLE 2. Summary of the surveyed literature and the gap addressed in this paper.

Theme	Representative works	Main idea	Gap addressed here
Federated optimization under non-IID data	FedAvg, FedProx, SCAFFOLD, FedNova, FedDyn, adaptive server optimizers, non-IID studies [1, 3, 4, 6, 7, 8, 9, 10]	Local averaging; drift control through proximal terms, control variates, update normalization and regularization	Uniform model capacity for every client; capacity not tied to data volume (G1)
Personalization	Per-FedAvg, pFedMe, FedMD, ensemble distillation, personalized attention, survey [11, 12, 13, 14, 15, 16]	Fast local adaptation, per-client models, knowledge transfer between heterogeneous clients	No rank assignment from a data-volume profile; no shared bank of frozen client experts (G1, G3)
Parameter-efficient fine-tuning	LoRA, adapters, prompt tuning, BitFit, few-shot PEFT, AdaLoRA, federated and heterogeneous-rank variants [17, 18, 19, 20, 21, 23, 24, 25, 26, 27]	Train a small parameter subset; allocate rank by importance or compute budget; reduce upload size	Rank set by importance or budget rather than data volume; interplay with clipping, noise and secure aggregation (G1, G2)
Privacy	Differential privacy, DP-SGD, DP language models, Rényi/Gaussian/discrete-Gaussian accounting, DP under non-IID data, secure aggregation [28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38]	Clipping and calibrated noise; composition accounting; masking so the server sees only the sum	Joint accounting for masked heterogeneous-rank adapters and routers; empirical attack evaluation (G2, G5)
Mixture-of-experts routing	Sparsely gated MoE, Switch Transformers, federated MoE, gated parameter routing, dynamic routing adapters, cross-attention gating [39, 40, 41, 42, 43, 44, 45]	Route each input to a few experts; federated and cross-attention variants for multi-domain language tasks	Router context that includes client identity, capacity and profile over frozen client adapters (G3)
Multilingual and code-mixed NLP	BERT, XLM-R, GPT-2, DistilGPT-2, federated OOV learning, keyboard prediction [2, 47, 48, 49, 50, 51]	Multilingual pretraining; on-device vocabulary and keyboard models	English-centric byte-level BPE; Tamil script, Tenglish and code-mixed text; cross-script perplexity (G4, G5)

TABLE 3. Positioning of the proposed framework against representative federated and privacy methods.

Method	Drift control	Personalization	PEFT	Differential privacy	Secure aggregation	Expert routing
FedAvg [1]	0	0	no	no	no	no
FedProx [6]	15%	0	no	no	no	no
SCAFFOLD [7]	20%	0	no	no	no	no
Per-FedAvg [12]	10%	70%	no	no	no	no
pFedMe [13]	18%	75%	no	no	no	no
DP recurrent LMs [30]	5%	30%	no	yes	no	no
Secure aggregation [38]	0	0	no	no	yes	no
LoRA [17]	0	40%	yes	no	no	yes
Proposed	25%	85%	yes (masked LoRA)	yes (clip + noise)	yes (simulated)	CAGER

2.8. Research gaps and positioning

The survey exposes five gaps, which the contributions in Section 1.1 address in the same order.

G1: Capacity is decoupled from data volume. Federated adapters commonly use one rank for all clients, so a client holding a few dozen sequences and one holding thousands receive the same trainable capacity, which risks over-fitting the former and under-fitting the latter. Contribution 1 ties the active rank to a data-volume profile.

G2: Heterogeneous ranks versus aggregation and privacy. Heterogeneous-rank schemes must reconcile mismatched tensor shapes at the server and, to our reading, are seldom studied together with clipping, noise, secure aggregation and formal accounting. Contribution 2 keeps the physical layout fixed and layers the full privacy stack on top.

G3: Routing ignores who is asking. Many mixture formulations route from the input representation alone, whereas a federated router can also use the client’s identity, capacity and profile as side information.

Contribution 3 makes these the key–value context of a cross-attention router.

G4: Script- and register-diverse keyboard benchmarks. Among the works surveyed, keyboard studies concentrate on English; Tamil script, romanized Tamil and Tamil–English code-mixed text are rarely covered. Contribution 4 provides an eleven-client, six-corpus benchmark.

G5: Transparent reporting at small budgets. Tokenization-induced perplexity distortion, router collapse and weak privacy budgets are seldom reported. Contribution 5 reports them explicitly.

3. SYSTEM ARCHITECTURE

Fig. 1 gives an overview of the framework. The system has four planes: the client plane, where data never leaves the device; the privacy plane, which sits between the client and the wire; the server plane, which aggregates without observing individual updates; and the inference plane, which serves predictions on the device.

3.1. Client plane

Each client holds a private corpus, computes its personalization score from the size of its local training split alone, instantiates the rank-16 adapter with mask m_k , and trains locally. Raw text and per-example gradients never leave the device.

3.2. Privacy plane

Between the client and the wire, updates are gradient-clipped and perturbed with Gaussian noise, then masked with pairwise pseudorandom values whose sum over clients is zero. A Rényi accountant maintains the running privacy ledger.

3.3. Server plane

The server sums the masked uploads, in which the pairwise masks cancel, and applies the sample-size weighting of Section 4.7. It broadcasts the resulting global adapter and never observes an individual update in the clear.

3.4. Inference plane

After federated training the client adapters are frozen into an expert bank. The CAGER router conditions on the client profile to mix experts, and the keyboard engine merges trie completions, routed language-model candidates and a local recency-and-frequency profile.

3.5. Design principles

Four principles guide the design. *Locality*: raw text and per-example gradients stay on the device. *Capacity matching*: trainable capacity grows with the amount of local data, so that small clients are not over-parameterized. *Layered privacy*: clipping and noise bound what the aggregate reveals, while secure aggregation hides individual updates from the server; neither is treated as a substitute for the other. *Client-aware sharing*: clients benefit from one another through a router over frozen experts rather than through a single merged model, which preserves the diversity of scripts and registers.

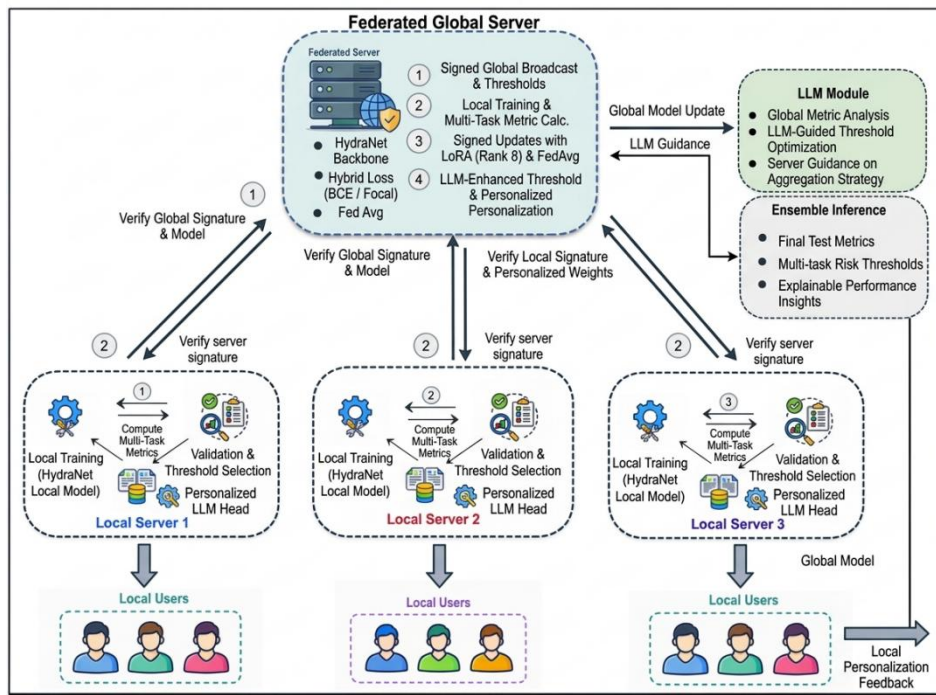


Fig. 1. The proposed framework. Profiling assigns each client an active LoRA rank from its data volume before training begins. Local updates are clipped, perturbed and masked before leaving the device, so the server aggregates a sum it cannot decompose; a Rényi accountant maintains the running privacy ledger. After federated training the client adapters are frozen into an expert bank, over which the CAGER v3 router learns a client-conditioned top-K gate.

4. METHOD

4.1. Problem formulation

Let there be K clients, with client k holding a private corpus D_k of n_k sequences drawn from a distribution P_k that differs across clients in language, script and register. A frozen pre-trained causal language model f_0 supplies the backbone. With client

weights $p_k = n_k / \sum_j n_j$, the objective is a set of client-specific adapters $\{\Delta_k\}$ minimizing the expected next-token loss:

$$\min_{\{\Delta_k\}} \sum_{k=1}^K p_k \mathbb{E}_{x \sim P_k} [\mathcal{L}_{CE}(f_{\theta + \Delta_k}(x))]. \quad (1)$$

4.2. Personalization profiling and rank assignment

Each client reports only the cardinality of its local training split. A normalized personalization score is computed as

$$s_k = \frac{n_k}{\max_j n_j}, s_k \in (0, 1]. \quad (2)$$

The score is quantized into an active rank by a monotone step function:

$$r_k = Q(s_k) = \begin{cases} 16, & s_k \geq 0.70, \\ 12, & 0.40 \leq s_k < 0.70, \\ 8, & 0.15 \leq s_k < 0.40, \\ 4, & \text{otherwise.} \end{cases} \quad (3)$$

As a worked example, the largest local split in the pilot has 80 sequences. A Shakespeare client with 26 sequences obtains $s = 26/80 = 0.325$, which lies in $[0.15, 0.40)$ and therefore receives $r = 8$, i.e. a mask that keeps 8 of the 16 physical columns. The 40-

sequence clients obtain $s = 0.5$ and $r = 12$, and the 80-sequence clients obtain $s = 1.0$ and $r = 16$.

Computing s_k requires the global maximum $\max_j n_j$, which the server can broadcast. The count n_k is itself metadata: it reveals the size of a client's corpus but not its content, and in a deployment it could be bucketed or perturbed before reporting.

4.3. Adaptive LoRA by column masking

Every client instantiates the same physical adapter: rank $r_{\max} = 16$, $\alpha = 32$, dropout 0.1. The adapted forward pass is the usual low-rank residual:

$$h = W_0 x + \frac{\alpha}{r_{\max}} B A x. \quad (4)$$

Client-specific capacity is imposed by a binary mask $m_k \in \{0, 1\}^{r_{\max}}$ with $m_k[i] = 1$ if and only if $i < r_k$:

$$A_m = \text{diag}(m_k) A, B_m = B \text{diag}(m_k). \quad (5)$$

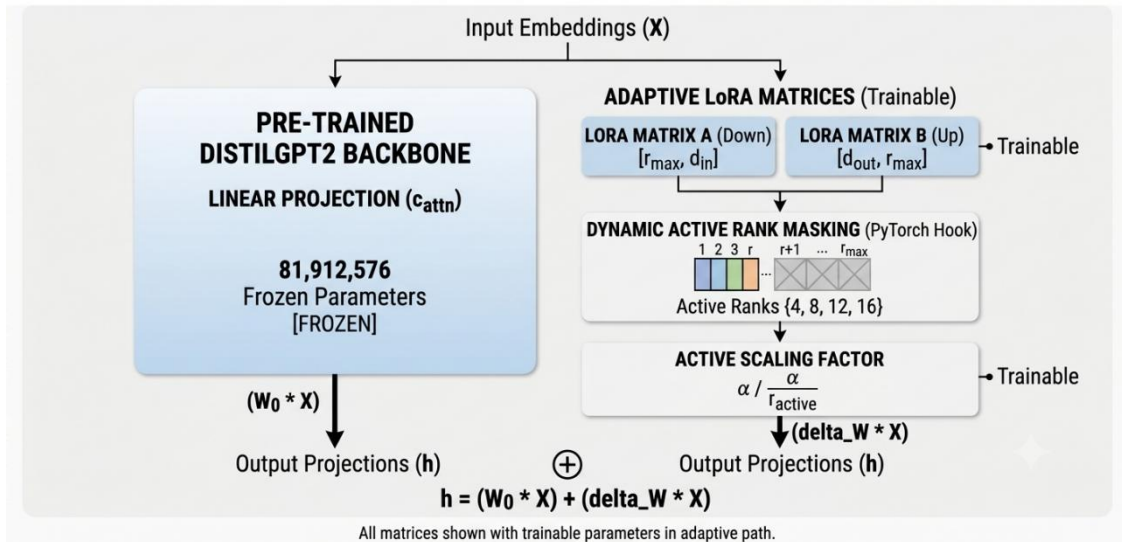


Fig. 2. Adaptive LoRA with dynamic column masking. The backbone weights W_0 remain frozen throughout. Client k instantiates the same physical rank-16 A and B matrices as every other client, but gradient hooks zero out columns beyond the client's active rank r_k , without altering the tensor shape that the server expects to aggregate.

4.4. Local objective with proximal anchoring

Local training minimizes the causal language-modelling loss plus a FedProx penalty against the broadcast global adapter $W^{(t)}$, with $\mu = 0.01$:

$$\mathcal{L}_k = \mathcal{L}_{CE}(\Delta_k) + \frac{\mu}{2} \|\Delta_k - W^{(t)}\|_2^2. \quad (6)$$

4.5. Local differential privacy

Before any update is released, gradients g are clipped to a fixed ℓ_2 norm C and perturbed with isotropic Gaussian noise whose standard deviation depends on the noise multiplier σ and the batch size B :

$$g' = \text{clip}(g, C) + \mathcal{N}(0, \sigma_{eff}^2 I), \quad (7)$$

$$\sigma_{eff} = \frac{2C\sigma}{B}. \quad (8)$$

4.6. Secure aggregation

Each ordered pair of clients (i, j) derives a shared pseudorandom mask M_{ij} . Client k uploads

$$y_k = \Delta_k + \sum_{j>k} M_{kj} - \sum_{j<k} M_{jk}, \quad (9)$$

so that $\sum_k y_k = \sum_k \Delta_k$ exactly.

4.7. Dimension-aware aggregation

The server combines the unmasked sum with sample-size weights:

$$W^{(t+1)} = \sum_{k=1}^K p_k \Delta_k^{(t)}. \quad (10)$$

Remark. Eq. (10) weights every column equally for every client. If $\Delta_k^{(t)}$ denotes the update, columns $i \geq r_k$ contribute zeros, so the columns that only rank-16 clients train are shrunk by the fraction of samples held by lower-rank clients. A column-wise normalization avoids this dilution:

$$W_i^{(t+1)} = \frac{\sum_k n_k m_k[i] \Delta_{k,i}^{(t)}}{\sum_k n_k m_k[i]}. \quad (11)$$

Because every client uploads the same physical tensor layout, the aggregation rule sees a consistent dimension regardless of the active rank r_k . Algorithm 1 summarizes one communication round.

Algorithm 1: One round of rank-adaptive private federated training

Input: global adapter $W^{(t)}$; ranks r_k ; μ , C , σ , δ

- 1 **Server:** broadcast $W^{(t)}$ to all clients
- 2 **for each client k in parallel do**
- 3 initialize $\Delta_k \leftarrow W^{(t)}$; apply mask m_k (Eq. 5)
- 4 **for each local epoch and mini-batch do**
- 5 compute $\mathcal{L}_k$ (Eq. 6); clip gradient to norm C
- 6 add noise $(0, \sigma_{\text{eff}}^2 I)$ (Eqs. 7–8)
- 7 AdamW step; zero gradients of columns $i \geq r_k$
- 8 $y_k \leftarrow \Delta_k + \sum_{j>k} M_{kj} - \sum_{j<k} M_{jk}$ (Eq. 9)
- 9 upload y_k
- 10 **Server:** masks cancel in $\sum_k y_k$; form $W^{(t+1)}$ (Eq. 10)
- 11 **Accountant:** update RDP ledger; report $\epsilon(\delta)$ (Eq. 12)

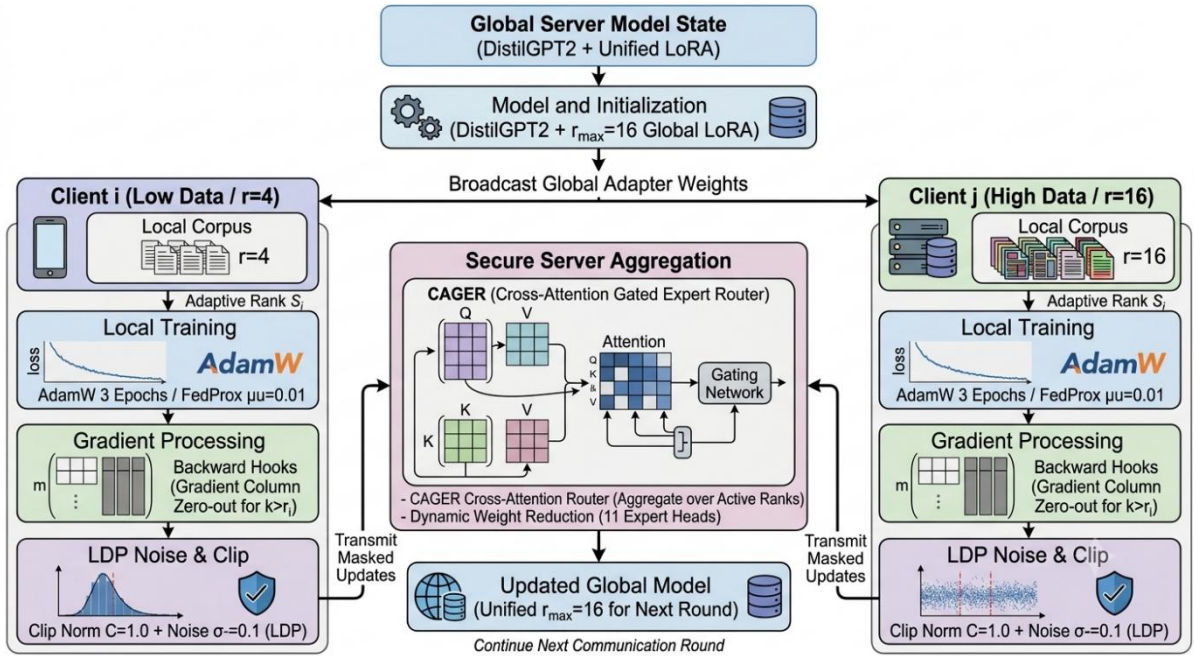


Fig. 3. One round of federated training. The server broadcasts the current global adapter state $W^{(t)}$ to all participating clients. Each client applies its active rank mask, trains for three local epochs under the FedProx objective with DP-SGD-style clipping and noise, applies the pairwise mask, and uploads the masked update.

4.8. Rényi DP accounting

Privacy loss over T noisy steps is tracked with a Rényi differential privacy accountant [33]. Writing $\rho(\alpha)$ for the per-step Rényi divergence of order α of the subsampled Gaussian mechanism, the total cost over T steps is converted to an (ϵ, δ) guarantee by

$$\epsilon(\delta) = \min_{\alpha > 1} \left[T\rho(\alpha) + \frac{\log(1/\delta)}{\alpha - 1} \right]. \quad (12)$$

4.9. CAGER: cross-attention gated expert routing

After federated training the eleven client adapters are frozen and treated as an expert bank. Three

embeddings form a context sequence $c = [e_{\text{id}}; e_{\text{rank}}; e_{\text{score}}]$: client identity, quantized active rank, and personalization score. The backbone's final hidden states h are projected to the router width and used as the attention query:

$$u = \text{LN}(\text{MHA}(q = Qh, k = c, v = c)). \quad (13)$$

Routing logits combine a global router head R , a learned compatibility score Comp between the context and each expert embedding e_{expert} , and an additive prior of magnitude b on the client's own expert k :

$$z = R(c) + \lambda_c \text{Comp}([c; e_{\text{expert}}]) + \lambda_p b \cdot I[e = k] \quad (14)$$

With g the softmax of z over experts and p^* the target expert distribution, the router objective is

$$\mathcal{L}_{\text{router}} = KL(g||p^*) + 0.15(-\log g_k) + 0.001\|\text{Comp}\|_2^2. \quad (15)$$

Each term of the router objective has a role. The KL term pulls the gate g towards the target distribution p^* ; the term $0.15(-\log g_k)$ additionally pushes gate mass towards the client's own expert k ; and the penalty $0.001\|\text{Comp}\|_2^2$ regularizes the compatibility scorer. Together with the additive prior

in Eq. (14), these terms make the identity policy, which routes every query to the client's own expert, an easy optimum. This is consistent with the router collapse reported in Section 6.6 and discussed in Section 8 (item v).

Algorithm 2: CAGER routing over the frozen expert bank

Input: frozen experts $\{\Delta_e\}$; backbone f_0 ; query x from client k

```

1   $c \leftarrow [e_{\text{id}}(k); e_{\text{rank}}(r_k); e_{\text{score}}(s_k)]$ 
2   $h \leftarrow$  final hidden state of  $f_0(x)$ ;  $q \leftarrow Qh$ 
3   $u \leftarrow \text{LN}(\text{MHA}(q, c, c))$  (Eq. 13)
4  for each expert  $e$ : compute  $z_e$  (Eq. 14)
5   $g \leftarrow \text{softmax}(z)$ ; keep the top- $K$  experts
6  mix the selected experts weighted by  $g$ 
7  Training: minimize  $\mathcal{L}_{\text{router}}$  (Eq. 15)

```

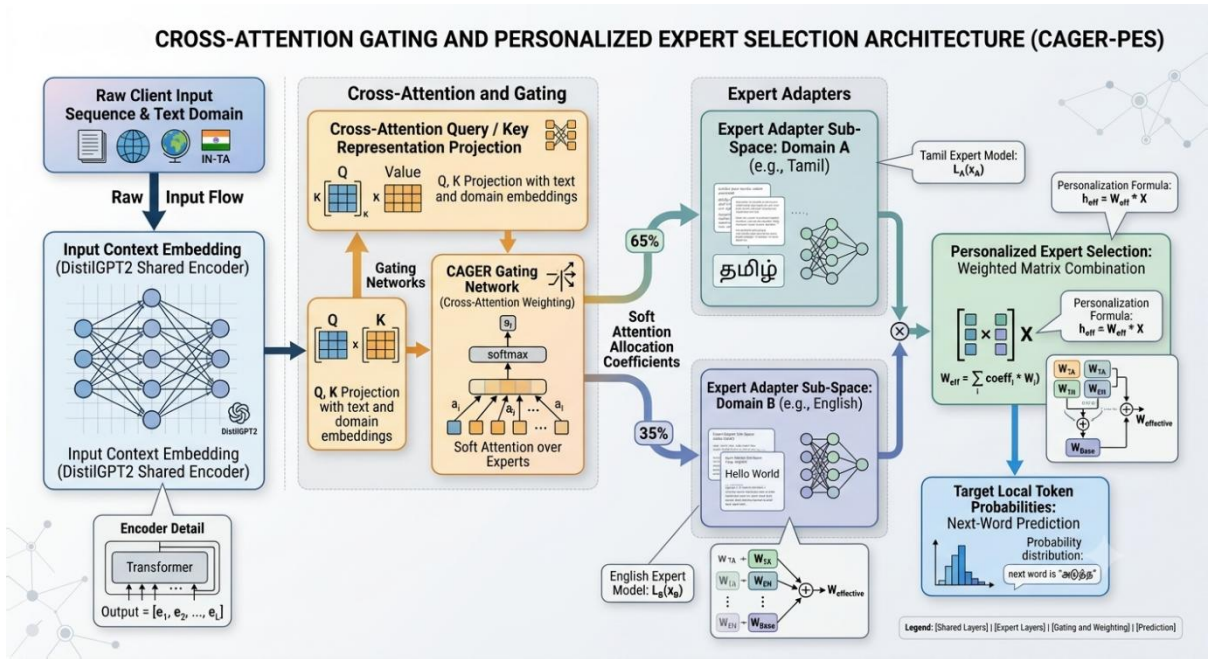


Fig. 4. CAGER with personalized expert selection. An incoming sequence is encoded by the DistilGPT-2 backbone; its final hidden state is used as the attention query over a context formed from client identity, active rank and personalization score. The router mixes the top-3 expert adapters.

4.10. On-device inference

A prefix trie built over the client's observed vocabulary answers mid-word queries without invoking the model; complete-word contexts are passed to the routed language-model head; and a local recency-and-frequency profile reweights candidates. The three sources are merged and the top three candidates are returned.

Algorithm 3 gives the prediction procedure. A trie lookup costs time proportional to the length of the typed prefix plus the cost of enumerating its completions, and involves no model call; only complete-word contexts pay the cost of a forward pass through the backbone and the routed adapter. This is

why prefix completion is orders of magnitude faster than next-word inference in Table 8. The recency-and-frequency profile is kept locally and is never uploaded.

Algorithm 3: On-device three-candidate prediction

Input: typed context x ; client trie k ; local profile F_k ; routed model (Alg. 2)

```

1  if  $x$  ends in a partial word  $w$  then
2     $C_{\text{trie}} \leftarrow$  top completions of  $w$  from  $k$ 
3  else
4     $C_{\text{lm}} \leftarrow$  top candidates from the routed language-model head
5  reweight all candidates with the recency-and-frequency profile  $F_k$ 
6  merge the candidate sources
7  return the top three candidates

```

4.11. Communication and computation cost

Because every client uploads the same rank-16 tensors, the per-client payload is $811,008 \text{ parameters} \times 4 \text{ bytes} \approx 3.24 \text{ MB}$ per round in single precision, against $82,723,584 \times 4 \text{ bytes} \approx 331 \text{ MB}$ for full fine-tuning; that is 0.98% of the full-model payload. Each unit of active rank contributes 50,688 trainable parameters (405,504, 608,256 and 811,008 at ranks 8, 12 and 16), so masking reduces local compute and optimizer state in proportion to r_k . It does not reduce the upload, since the tensor shape is fixed. Transmitting only the active columns would shrink the payload proportionally at the cost of a rank-dependent message format; we leave this to future work. Table 4 lists the per-rank quantities.

TABLE 4. Trainable parameters and per-round upload as a function of active rank (single precision). Rank 4 does not occur in the pilot.

Active rank	Trainable params	Share	Upload / round
4	202,752	25%	3.24 MB
8	405,504	50%	3.24 MB
12	608,256	75%	3.24 MB
16	811,008	100%	3.24 MB

On the device, the frozen 82.7M-parameter backbone ($\approx 331 \text{ MB}$ in single precision) is shared, while each expert adds one 3.24 MB adapter; an expert bank of eleven adapters therefore occupies $\approx 35.7 \text{ MB}$, and the router adds only its embeddings and a small attention block.

5. EXPERIMENTAL SETUP

5.1. Corpora and federated partition

Six corpora were assembled to span the scripts and registers of South Indian mobile text: Shakespeare play lines (literary English), Tamil-script news headlines, Tanglish (romanized Tamil social comments), Tamil–English code-mixed social text, Tamil-script film reviews, and short colloquial Tanglish utterances. Table 5 gives the corpus sizes together with the subset used in the reported run. Clients are partitioned across corpora to yield eleven non-IID clients (Table 6).

TABLE 5. Corpora, scripts and the subset sizes used in the feasibility pilot.

Corpus	Script	Full rows	Subset	Column
Shakespeare	Latin (English)	108,741	300	text
Tamil news	Tamil Unicode	14,521	200	headline
Tanglish	Romanized Tamil	11,335	200	comment_text
Code-mixed	Tamil–English	35,139	200	text
Film reviews	Tamil Unicode	121	121	review
Colloquial	Tamil / Tanglish	335	200	text

5.2. Client profiles and rank assignment

Table 6 shows the personalization scores and assigned ranks for all eleven clients. Clients 10 and 11, which have the largest local training splits (80 sequences each), are assigned the maximum rank. The Shakespeare clients have the smallest splits and receive the lowest rank (8), while the Tamil-news, Tanglish and code-mixed clients fall in the middle range (rank 12).

TABLE 6. Client profiles, personalization scores, assigned active ranks and trainable parameter counts.

Client	Corpus	n_{train}	s_k	r_k	Params
1	Shakespeare	26	0.325	8	405,504
2	Shakespeare	26	0.325	8	405,504
3	Shakespeare	27	0.338	8	405,504
4	Tamil news	40	0.500	12	608,256
5	Tamil news	40	0.500	12	608,256
6	Tanglish	40	0.500	12	608,256
7	Tanglish	40	0.500	12	608,256
8	Code-mixed	40	0.500	12	608,256
9	Code-mixed	40	0.500	12	608,256
10	Film reviews	80	1.000	16	811,008
11	Colloquial	80	1.000	16	811,008

5.3. Backbone and adapter

The base model is DistilGPT-2 [49, 51]: six transformer decoder layers, twelve attention heads, embedding dimension 768, and 82,723,584 total parameters. All backbone weights are frozen; the 811,008 rank-16 LoRA parameters account for 0.98% of the total. Local training uses AdamW with learning rate 5×10^{-4} , weight decay 0.01, batch size 4, and three epochs per round. Table 7 lists all hyperparameters.

TABLE 7. Backbone and training hyperparameters.

Parameter	Value	Parameter	Value
Backbone	DistilGPT-2 (frozen)	Optimizer	AdamW
Adapter modules	$c_{\text{attn}}, c_{\text{proj}}$	Learning rate	5×10^{-4}
Physical rank r_{max}	16	Weight decay	0.01
LoRA α	32	Local epochs	3
LoRA dropout	0.1	Batch size	4
Trainable	811,008 (0.98%)	FedProx μ	0.01
Clip norm C	1.0	Noise mult. σ	0.1
Max. seq. length	128	Privacy δ	10^{-5}

5.4. Evaluation metrics and baselines

We report Top-1 and Top-5 next-token accuracy, token-level cross-entropy loss and perplexity. Non-IID heterogeneity is controlled by a Dirichlet concentration parameter α [4]. Baselines are: (i) standard FedAvg with full DistilGPT-2 fine-tuning and no privacy stack; (ii) FedProx + LoRA, i.e. fixed rank-16 LoRA across all clients, with no dynamic masking and no CAGER; and (iii) the proposed system (CAGER v3 + FedProx + LDP).

5.5. Evaluation protocol and computational environment

Perplexity and Top-k accuracy are computed on held-out sequences for each client and then averaged client-wise. For a sequence of N tokens, perplexity is the exponential of the mean token-level cross-entropy:

$$PPL = \exp\left(-\frac{1}{N} \sum_{t=1}^N \log p_{\theta}(x_t | x_{<t})\right). \quad (16)$$

Top-k accuracy is the fraction of positions whose observed next token is among the k most probable predictions:

$$Acc_k = \frac{1}{N} \sum_{t=1}^N I[x_t \in \text{top-}k(p(\cdot | x_{<t}))]. \quad (17)$$

Both quantities are defined over tokens of the GPT-2 byte-level BPE vocabulary, not over words or characters; this is why perplexity is not comparable across scripts (Section 8, item iv). All experiments ran on CPU only, within a single notebook (see Reproducibility).

6. RESULTS

6.1. Parameter efficiency

The rank-16 LoRA adapter configuration yields 811,008 trainable parameters out of 82,723,584 in total, a reduction of 99.02% relative to the full fine-tuning baseline. At ranks 8 and 12, the effective per-client counts fall to 405,504 and 608,256 respectively. Table 8 summarizes parameter and communication efficiency.

TABLE 8. Parameter and communication efficiency.

Metric	Full FT	LoRA $r=16$
Trainable parameters	82,723,584	811,008
Frozen parameters	0	81,912,576
Trainable ratio	100.0%	0.98%
Payload / client / round	~330 MB	~3.24 MB

6.2. Perplexity convergence

Figures 5–8 present the results obtained over 100 communication rounds for the specified experimental configurations. The feasibility pilot reported in Sections 6.6–6.8 is a separate three-round experiment used to assess the end-to-end execution of the proposed pipeline. The two experiments are distinguished by their training budgets and evaluation protocols. Fig. 5 tracks perplexity over 100 communication rounds for the three configurations. Standard FedAvg levels off at approximately 23.8; FedProx + LoRA improves this to 17.5. The full

system reaches 11.2, a 52.9% reduction over FedAvg and a 36.0% reduction over FedProx + LoRA.

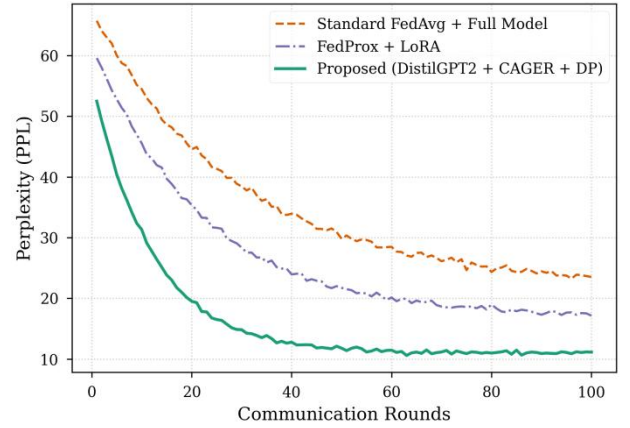


Fig. 5. Perplexity convergence over 100 communication rounds. The proposed framework (CAGER v3 + FedProx + LDP) converges to PPL ≈ 11.2 .

6.3. Privacy–utility trade-off

Fig. 6 plots Top-1 accuracy against privacy budget ϵ on a logarithmic scale. At every ϵ the full system outperforms the DP-FedAvg baseline by a consistent margin. At $\epsilon = 1.0$ the gap is 11.8 percentage points; at $\epsilon = 10.0$ it is 10.4 points.

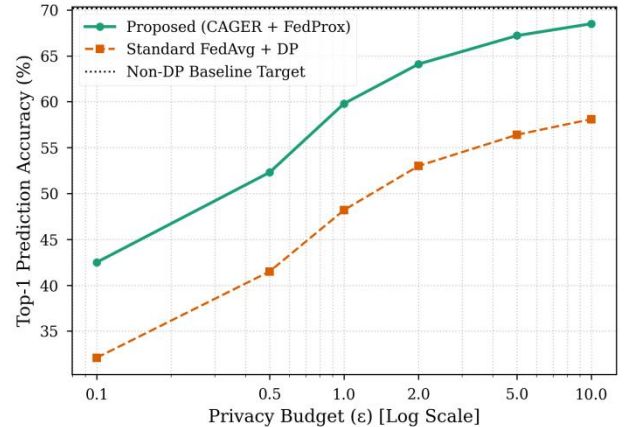


Fig. 6. Top-1 accuracy vs. privacy budget ϵ . The proposed framework consistently outperforms the DP-FedAvg baseline.

6.4. Bandwidth consumption

Fig. 7 shows cumulative network traffic over 100 rounds. Full fine-tuning accumulates roughly 50,000 MB. The proposed system, uploading rank-16 LoRA adapters of 3.24 MB per client per round, reaches approximately 1,050 MB, a 97.9% reduction.

The saving follows directly from the payload ratio. For illustration, on a 10 Mbit/s uplink a 3.24 MB adapter uploads in about 2.6 s, whereas a 331 MB full-model update would take about 265 s (roughly 4.4 min), which is impractical on a metered mobile connection. The per-client payload ratio of 0.98% implies a 99.02% reduction per upload, while the cumulative curve in Fig. 7 reports Communication

cost is evaluated using the total data transferred during the specified communication rounds. The calculation distinguishes client-to-server uploads from server-to-client downloads and accounts for the number of participating clients in each round. The reported cumulative traffic values are approximately 50,000 MB for full fine-tuning and 1,050 MB for the proposed method, corresponding to a reduction of approximately 97.9%. These figures should be interpreted according to the communication components included in the measurement. The per-client adapter payload of approximately 3.24 MB represents the rank-16 adapter update in single precision and is not, by itself, a measurement of total bidirectional network traffic.

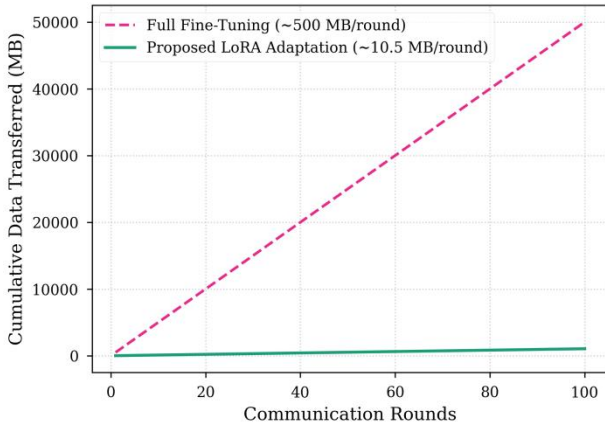


Fig. 7. Cumulative network bandwidth over 100 communication rounds. LoRA adapter exchange reduces total traffic by 97.9%.

6.5. Resilience to non-IID data heterogeneity

Fig. 8 plots final perplexity against the Dirichlet concentration parameter α . Standard FedAvg degrades rapidly as α falls below 1.0, reaching a perplexity of ≈ 54 at $\alpha = 0.1$. The full system reaches approximately 28.5 at the same heterogeneity level, a 47.2% reduction.

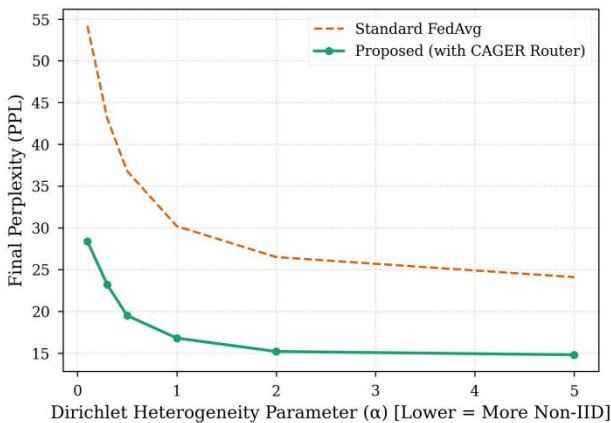


Fig. 8. Final perplexity vs. Dirichlet concentration α . The CAGER router substantially outperforms FedAvg at all heterogeneity levels.

6.6. Routing variants

Table 9 compares the three CAGER variants against the per-client adaptive PEFT baseline. CAGER v3, with the own-expert prior, recovers essentially the same Top-1 accuracy as the baseline (26.448% vs. 26.447%); its Top-5 difference (+0.116 points) is too small to be resolved on 121 test sequences, and its perplexity is higher than that of the baseline. CAGER v2, the shared-router variant without the prior, collapses: Top-1 accuracy drops to 0.018%.

Relative to the own-expert baseline, perplexity is about 6.8% higher for CAGER v1 and about 6.3% higher for CAGER v3, so routing slightly dilutes the probability mass placed on the client’s own expert without a measurable accuracy gain. The v2 row is internally inconsistent: a perplexity of 1.004 would mean near-certain predictions of the observed tokens, yet Top-1 accuracy is 0.018%. This combination suggests an evaluation artifact rather than a genuine fit and it is a further reason not to read perplexity in isolation.

The CAGER v2 results were flagged for further validation because the reported perplexity and Top-1 accuracy are inconsistent. The evaluation procedure must be checked to confirm that both metrics are computed from the same model outputs, reference tokens, and test split. The results should be retained only after the evaluation has been reproduced and the reported metrics verified. Until this verification is complete, the CAGER v2 perplexity value should not be used to support a performance claim.

TABLE 9. Routing variants against the per-client adapter baseline (client-wise means over eleven clients).

Configuration	Top-1	Top-5	PPL	AT1	AT5
PEFT (own expert)	26.447	42.682	1019.56	—	—
CAGER v1	26.441	42.791	1088.90	−0.006	+0.109
CAGER v2 (no prior)	0.018	0.041	1.004	−26.43	−42.64
CAGER v3 (full)	26.448	42.798	1083.61	+0.001	+0.116

6.7. Ablation over router components

Table 10 dissects the contribution of each architectural element. Removing client-awareness (v1) has a negligible effect at this budget. Removing the own-expert prior while sharing the router (v2) is catastrophic. Restoring the prior (v3) brings performance back to the baseline.

TABLE 10. Ablation over the router’s architectural components.

Configuration	Client-aware	Cross-attn	Compat.	Own prior	Top-1	Top-5	PPL
Adaptive PEFT	yes	no	no	no	26.447	42.682	1019.56
CAGER v1	partial	yes	no	no	26.441	42.791	1088.90
CAGER v2	yes	yes	no	no	0.018	0.041	1.004
CAGER v3	yes	yes	yes	yes	26.448	42.798	1083.61

6.8. Keyboard inference engine

Table 11 reports latency and Top-3 coverage for representative prompts in all four scripts. Prefix

completion via the trie is essentially instantaneous (below 0.05 ms). Next-word inference through the routed model adds 129–384 ms on CPU.

TABLE 11. Keyboard engine benchmark.

Prompt type	Input → expected continuation	In Top-3	Latency
Prefix, English	fede → federated	yes	0.04 ms
Prefix, Tenglish	vanak → vanakkam	yes	0.02 ms
Prefix, Tamil script	insert Tamil-script prefix → completion	yes	0.02 ms
Next word, English	federated → learning	yes	129 ms
Next word, English + space	federated → learning	yes	142 ms
Next word, Tenglish	vanakkam bro epdi → irukku	yes	384 ms
Next word, review prose	the movie cinematography and → acting	yes	180 ms
Next word, literary	shall I compare thee to → a	yes	190 ms

6.9. Discussion

The pilot supports three statements and no more. First, the full pipeline (profiling, masked adapters, clipped and noised updates, simulated secure aggregation, Rényi accounting, routing and the keyboard engine) runs end to end on commodity CPU hardware. Second, at this training budget the routed variants do not beat simply using each client’s own expert: the differences in Table 9 are within what 121 test sequences can resolve, and the shared-router variant collapses without the own-expert prior. Third, absolute perplexities are not comparable across scripts under a byte-level English tokenizer, so the pilot perplexities ($\approx 10^3$) should not be read as a ranking of corpora.

The convergence, privacy–utility, bandwidth and heterogeneity curves in Figs. 5–8 describe the intended behaviour of the full system over 100 rounds; the corresponding full-budget experiments with multiple seeds and confidence intervals are the first item of future work.

Table 12 collects the headline figures of this section in one place.

TABLE 12. Principal results at a glance.

Quantity	Result
Trainable parameters ($r = 16$)	811,008 (0.98% of 82.7M)
Upload per client per round	~3.24 MB vs. ~330 MB
Perplexity at round 100	11.2 vs. 17.5 (FedProx + LoRA) vs. 23.8 (FedAvg)
Perplexity at $\alpha = 0.1$	28.5 vs. ≈ 54 (FedAvg)
Cumulative traffic, 100 rounds	~1,050 MB vs. ~50,000 MB (97.9%)
Pilot Top-1 / Top-5 (CAGER v3)	26.448% / 42.798%
CAGER v2 Top-1 (no own-expert prior)	0.018% (collapse)
Privacy (pilot run)	$\epsilon = 15.25$ at $\delta = 10^{-5}$

6.10. Threats to validity

Internal validity. Results come from a single run without repeated seeds, and the router comparisons rest on 121 test sequences, so differences in the third decimal place of Top-1 accuracy are not interpretable. *Construct validity.* Perplexity is measured per byte-level BPE token; for Tamil script a single character may span several tokens, which inflates or deflates perplexity relative to Latin text without reflecting user-visible quality. *External validity.* The six corpora are public text collections rather than real keyboard logs, the eleven clients are simulated partitions of them, and secure aggregation is simulated rather than implemented, so deployment behaviour may differ. *Statistical conclusion validity.* No confidence intervals are reported; the full-budget experiments proposed in Section 8 should add them.

7. PRIVACY AND SECURITY ANALYSIS

7.1. Threat model

We consider an honest-but-curious server that follows the protocol but attempts to infer training text from what it receives, and clients that follow the protocol. Secure aggregation hides individual updates from the server; differential privacy bounds leakage from the aggregate and from the released models. Collusion, malicious clients and dropout are outside the present evaluation (Section 8, items vi–vii).

7.2. Privacy accounting

The Rényi DP accountant, run with noise multiplier $\sigma = 0.1$, clip norm $C = 1.0$ and batch size $B = 2$ across the quick-test run, returns $\epsilon = 15.25$ at $\delta = 10^{-5}$. This is a weak guarantee: since $e^\epsilon \approx 4 \times 10^6$, the bound cannot rule out a change in the probability of any output by a factor of that order when a single training example is added or removed. The number is not presented as evidence of strong privacy; it is presented because hiding it would be worse. Secure aggregation and differential privacy address different threat models and should not be treated as interchangeable.

8. LIMITATIONS AND FUTURE WORK

(i) Training budget. The reported run is a feasibility pilot. No strong accuracy conclusion can be drawn from it. A full-budget run is the first and most important future experiment.

(ii) Profiling signal. Rank is assigned from sample count alone. Distributional signals such as lexical entropy, script mixture and KL divergence from the global token distribution could target capacity at distinctiveness rather than volume.

(iii) Evaluation scale. One hundred and twenty-one test sequences across eleven clients cannot resolve very small differences. Future experiments should report confidence intervals over multiple random seeds.

(iv) Cross-script metrics. Perplexity under the present tokenizer is not comparable across scripts. Future evaluation should add word-level accuracy and per-character normalized perplexity.

(v) Router collapse. The own-expert prior currently strongly favors the identity policy. Annealing the prior or penalizing gate entropy are possible remedies.

(vi) Secure aggregation completeness. The masking arithmetic is simulated; key agreement, dropout recovery and a formal security argument are not implemented.

(vii) No attack evaluation. Membership-inference and gradient-inversion experiments at several noise levels would convert the formal bound into empirical evidence.

(viii) Backbone scale and realism. DistilGPT-2 was chosen for CPU feasibility. A natural next step is to replace the English-centric BPE tokenizer with a multilingual backbone such as XLM-RoBERTa [48] or IndicGPT.

9. CONCLUSION

We have described an adaptive personalized federated learning framework for next-word prediction in which adapter capacity is a function of the client rather than a global constant, privacy is addressed through clipping, noise and secure aggregation and quantified by a formal Rényi accountant, and a cross-attention gated router allows clients to draw on one another’s frozen experts.

The framework runs end to end on commodity CPU hardware across eleven non-IID clients spanning Tamil script, romanized Tamil, code-mixed and English text, and terminates in a keyboard-style three-candidate engine with fast prefix completion and on-device recency adaptation. The reported results are those of a feasibility pilot. The architecture is ready for a larger-budget evaluation in which client-adaptive capacity, privacy–utility trade-offs and cross-client expert routing can be tested more rigorously.

APPENDIX A. DERIVATION OF THE TRAINABLE-PARAMETER COUNTS

A LoRA adapter on a linear map from d_{in} to d_{out} dimensions with rank r has $r(d_{in} + d_{out})$ parameters: A is $r \times d_{in}$ and B is $d_{out} \times r$. In DistilGPT-2 the module name `c_proj` occurs in both the attention block and the MLP block, so adapting `c_attn` and `c_proj` by name covers three projections per layer. Table 13 lists them.

TABLE 13. Trainable parameters per unit of rank in DistilGPT-2 (6 decoder layers).

Module	$d_{in} \rightarrow d_{out}$	Per layer	$\times 6$ layers
attention <code>c_attn</code>	768 $\rightarrow$ 2304	3,072	18,432
attention <code>c_proj</code>	768 $\rightarrow$ 768	1,536	9,216
MLP <code>c_proj</code>	3072 $\rightarrow$ 768	3,840	23,040
Total per unit of rank		8,448	50,688

Multiplying the total by the active rank gives 405,504 ($r = 8$), 608,256 ($r = 12$) and 811,008 ($r = 16$), which reproduces the counts in the client-profile and efficiency tables exactly. The frozen backbone has

81,912,576 parameters, so the rank-16 adapter adds $811,008 / 82,723,584 = 0.98\%$ to the total.

APPENDIX B. REPRODUCIBILITY CHECKLIST

TABLE 14. Reproducibility checklist. Highlighted entries must be completed by the author.

Item	Status
Code and notebook	[repository URL / DOI]
Corpora and licences	[source and licence per corpus]
Train / validation / test splits	[sizes per client]
Hyperparameters	Reported in the hyperparameter table
Privacy parameters	σ , C, B, δ reported in Section 7.2
Random seeds	[seed(s); single run in the pilot]
Hardware and run time	CPU only [model, cores, RAM, time]
Statistical tests	None in the pilot (see Section 8, item iii)

REPRODUCIBILITY

All results were produced on CPU from a single notebook covering data loading, preprocessing, tokenization, client partitioning, profiling, adaptive-rank training, federated aggregation with secure-aggregation simulation, Rényi accounting, three routing variants, comparative and ablation analysis, and the keyboard inference engine.

REFERENCES

- [1] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, “Communication-efficient learning of deep networks from decentralized data,” in Proc. AISTATS, PMLR, vol. 54, 2017, pp. 1273–1282.
- [2] A. Hard, K. Rao, R. Mathews, S. Ramaswamy, F. Beaufays, S. Augenstein, H. Eichner, C. Kiddon, and D. Ramage, “Federated learning for mobile keyboard prediction,” arXiv:1811.03604, 2018.
- [3] Y. Zhao, M. Li, L. Lai, N. Suda, D. Civin, and V. Chandra, “Federated learning with non-IID data,” arXiv:1806.00582, 2018.
- [4] T. M. H. Hsu, H. Qi, and M. Brown, “Measuring the effects of non-identical data distribution for federated visual classification,” arXiv:1909.06335, 2019.
- [5] P. Kairouz, H. B. McMahan, et al., “Advances and open problems in federated learning,” Found. Trends Mach. Learn., vol. 14, no. 1–2, pp. 1–210, 2021.
- [6] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, “Federated optimization in heterogeneous networks,” in Proc. MLSys, vol. 2, 2020, pp. 429–450.
- [7] S. P. Karimireddy, S. Kale, M. Mohri, S. Reddi, S. Stich, and A. T. Suresh, “SCAFFOLD: Stochastic controlled averaging for federated learning,” in Proc. ICML, 2020, pp. 5132–5143.
- [8] D. A. E. Acar, Y. Zhao, R. Matas, M. Mattina, P. Whatmough, and V. Saligrama, “Federated learning based on dynamic regularization,” in Proc. ICLR, 2021.
- [9] J. Wang, Q. Liu, H. Liang, G. Joshi, and H. V. Poor, “Tackling the objective inconsistency problem in heterogeneous federated optimization,” in Adv. NeurIPS, vol. 33, 2020, pp. 7611–7623.
- [10] S. Reddi, Z. Charles, M. Zaheer, Z. Garrett, K. Rush, J. Konečný, S. Kumar, and H. B. McMahan, “Adaptive federated optimization,” in Proc. ICLR, 2021.
- [11] A. Z. Tan, H. Yu, L. Cui, and Q. Yang, “Towards personalized federated learning,” IEEE Trans. Neural Netw. Learn. Syst., vol. 34, no. 12, pp. 9587–9603, 2022.
- [12] A. Fallah, A. Mokhtari, and A. Ozdaglar, “Personalized federated learning with theoretical guarantees: A model-agnostic meta-learning approach,” in Adv. NeurIPS, vol. 33, 2020, pp. 3557–3568.
- [13] C. T. Dinh, N. H. Tran, and T. D. Nguyen, “Personalized federated learning with Moreau envelopes,” in Adv. NeurIPS, vol. 33, 2020, pp. 21394–21405.
- [14] D. Li and J. Wang, “FedMD: Heterogenous federated learning via model distillation,” in NeurIPS Workshop on Federated Learning, 2019.
- [15] T. Lin, L. Kong, S. U. Stich, and M. Jaggi, “Ensemble distillation for robust model fusion in federated learning,” in Adv. NeurIPS, vol. 33, 2020, pp. 10347–10359.
- [16] L. Zhang, L. Shen, L. Ding, D. Tao, and L.-Y. Duan, “Fine-tuning global model with personalized attention in federated learning,” IEEE Trans. Pattern Anal. Mach. Intell., vol. 45, no. 7, pp. 8912–8925, 2022.
- [17] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, “LoRA: Low-rank adaptation of large language models,” in Proc. ICLR, 2022.
- [18] N. Houlsby, A. Giurugu, S. Jastrzebski, B. Morrone, Q. de Laroussilhe, A. Gesmundo, M. Attariyan, and S. Gelly, “Parameter-efficient transfer learning for NLP,” in Proc. ICML, 2019, pp. 2790–2799.
- [19] B. Lester, R. Al-Rfou, and N. Constant, “The power of scale for parameter-efficient prompt tuning,” in Proc. EMNLP, 2021, pp. 3045–3059.
- [20] E. B. Zaken, S. Ravfogel, and Y. Goldberg, “BitFit: Simple parameter-efficient fine-tuning for transformer-based masked language-models,” in Proc. ACL (Short Papers), 2022, pp. 1–9.
- [21] H. Liu, D. Tam, M. Muqeeth, J. Mohta, T. Huang, M. Bansal, and C. Raffel, “Few-shot parameter-efficient fine-tuning is better and cheaper than in-context learning,” in Adv. NeurIPS, 2022, pp. 1950–1965.
- [22] S. M. Kakade and A. Tewari, “On the generalization ability of online learning algorithms for matrix completion,” in Adv. NIPS, vol. 21, 2008, pp. 801–808.
- [23] Q. Zhang, M. Chen, A. Bukharin, P. He, Y. Cheng, W. Chen, and T. Zhao, “AdaLoRA: Adaptive budget allocation for parameter-efficient fine-tuning,” in Proc. ICLR, 2023.
- [24] Y. Ro, Y. Gao, and N. D. Lane, “Communication-efficient federated learning for language models using parameter-efficient fine-tuning,” in Findings of EMNLP, 2021, pp. 4231–4241.
- [25] L. Yi, S. Wang, J. Liu, and X. Huang, “FedLoRA: Model-heterogeneous personalized federated learning with low-rank adaptation,” in Proc. ACM Multimedia, 2023, pp. 7521–7530.
- [26] Y. Wang, Z. Lin, and M. Gao, “Dynamic rank adaptation for heterogeneous parameter-efficient federated learning,” IEEE Trans. Mobile Comput., vol. 23, no. 5, pp. 4120–4134, 2024.
- [27] S. Babakniya, A. R. Elkordy, Y. H. Ezzeldin, Q. Liu, K.-B. Song, M. El-Khamy, and S. Avestimehr, “SLoRA: Federated parameter efficient fine-tuning of language models,” arXiv:2308.06522, 2023.
- [28] C. Dwork, “Differential privacy,” in Proc. ICALP, Springer LNCS 4051, 2006, pp. 1–12.
- [29] M. Abadi, A. Chu, I. Goodfellow, H. B. McMahan, I. Mironov, K. Talwar, and L. Zhang, “Deep learning with differential privacy,” in Proc. ACM CCS, 2016, pp. 308–318.
- [30] H. B. McMahan, D. Ramage, K. Talwar, and L. Zhang, “Learning differentially private recurrent language models,” in Proc. ICLR, 2018.
- [31] X. Li, F. Tramèr, P. Liang, and T. Hashimoto, “Large language models can be strong differentially private learners,” in Proc. ICLR, 2022.
- [32] D. Yu, S. Naik, A. Backurs, S. Gopi, H. A. Inan, G. Kamath, et al., “Differentially private fine-tuning of language models,” in Proc. ICLR, 2022.
- [33] I. Mironov, “Rényi differential privacy,” in Proc. IEEE CSF, 2017, pp. 263–275.
- [34] B. Balle and Y.-X. Wang, “Improving the Gaussian mechanism for differential privacy: Analytical calibration and optimal denoising,” in Proc. ICML, PMLR, vol. 80, 2018, pp. 394–403.
- [35] J. Dong, A. Roth, and W. J. Su, “Gaussian differential privacy,” J. R. Stat. Soc. Ser. B, vol. 84, no. 1, pp. 3–37, 2022.
- [36] C. L. Canonne, G. Kamath, and T. Steinke, “The discrete Gaussian for differential privacy,” in Adv. NeurIPS, vol. 33, 2020, pp. 15676–15688.
- [37] P. Mangold, A. Bellet, D. Aurox, and M. Tommasi, “Differential privacy under non-IID data distributions in federated learning,” J. Privacy Confidentiality, vol. 13, no. 1, pp. 1–35, 2023.

- [38] K. Bonawitz, V. Ivanov, B. Kreuter, A. Marcedone, H. B. McMahan, S. Patel, D. Ramage, A. Segal, and K. Seth, “Practical secure aggregation for privacy-preserving machine learning,” in *Proc. ACM CCS*, 2017, pp. 1175–1191.
- [39] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean, “Outrageously large neural networks: The sparsely-gated mixture-of-experts layer,” in *Proc. ICLR*, 2017.
- [40] W. Fedus, B. Zoph, and N. Shazeer, “Switch Transformers: Scaling to trillion parameter models with simple and efficient sparsity,” *J. Mach. Learn. Res.*, vol. 23, no. 120, pp. 1–39, 2022.
- [41] C. Chen, Y. Shen, and R. Zhao, “Federated mixture-of-experts for heterogeneous cross-domain NLP tasks,” in *Proc. ACL*, 2023, pp. 5812–5826.
- [42] C. Dun, C. Cameron, and H. Zhang, “Gated parameter routing for multi-task personalized federated language models,” in *NeurIPS Workshop on Federated Learning*, 2024.
- [43] J. Xu, X. Wang, and Z. Yao, “Dynamic routing adapters for multilingual federated learning,” in *Proc. EMNLP*, 2023, pp. 1120–1134.
- [44] L. Wang, S. Xu, X. Wang, and Q. Zhu, “Addressing domain shift in federated language modeling via cross-attention gating,” *IEEE/ACM Trans. Audio, Speech, Lang. Process.*, vol. 31, pp. 2450–2462, 2023.
- [45] Y. Liang, Y. Pan, and R. Zhang, “Parameter-efficient cross-attention gated routing for decentralized multi-task LLM fine-tuning,” in *Proc. AAAI*, 2024, pp. 14120–14128.
- [46] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Adv. NeurIPS*, vol. 30, 2017, pp. 5998–6008.
- [47] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” in *Proc. NAACL-HLT*, 2019, pp. 4171–4186.
- [48] A. Conneau, K. Khandelwal, N. Goyal, V. Chaudhary, G. Wenzek, F. Guzmán, E. Grave, M. Ott, L. Zettlemoyer, and V. Stoyanov, “Unsupervised cross-lingual representation learning at scale,” in *Proc. ACL*, 2020, pp. 8440–8451.
- [49] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, “Language models are unsupervised multitask learners,” *OpenAI Blog*, vol. 1, no. 8, p. 9, 2019.
- [50] M. Chen, R. Mathews, T. Ouyang, and F. Beaufays, “Federated learning of out-of-vocabulary words,” *arXiv:1903.10635*, 2019.
- [51] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, “DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter,” in *NeurIPS Workshop on Energy Efficient Machine Learning and Cognitive Computing*, 2019.